

# Современные методы оптимизации

Часть 1: XX век

К началу XIX века:

Найти экстремум:

1. Решить  $f'(x) = 0$
2. Проверить  $f''(x)$

**НО:** большинство уравнений  
не решаются аналитически!

- ▶ Полиномы  $\geq 5$
- ▶ Трансцендентные ур-я

## Пример

Минимум:

$$f(x) = x^5 - 3x^3$$

$$f'(x) = 5x^4 - 9x^2 = 0$$

**Нет формулы!**

Решение: итеративное приближение

# Интуиция: спуск с холма

Ситуация:

Вы на холме в тумане.

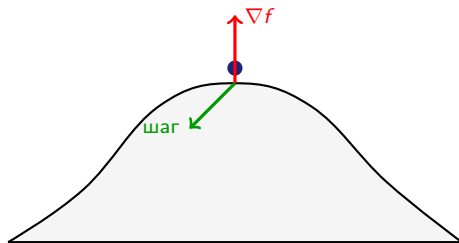
Можете:

- ▶ Определить наклон
- ▶ Понять, где “вниз”

**Вопрос:** Куда идти?

**Ответ:**

Против градиента!



Градиент  $\uparrow$  вверх, шаг  $\downarrow$  вниз

Градиент — компас вверх

## Шаги:

1. Начать с  $x_0$
2. Вычислить  $g = \nabla f(x_t)$
3. Шаг:  $x_{t+1} = x_t - \alpha g$
4. Повторять

## Learning rate $\alpha$

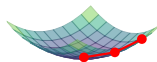
- ▶ Большой  $\rightarrow$  прыгаем
- ▶ Маленький  $\rightarrow$  медленно
- ▶ Оптимальный  $\rightarrow$  ОК

## Формулы

$$x_0 = \text{random}$$

$$g = \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha g$$



Приближаемся шаг за шагом

До 1847

**Аналитика**

- ✓ Точное решение
- ✗ Только простые

После 1847

**Алгоритмы**

- ✓ Итеративно
- ✓ Любые функции

Коши

**Огюстен Коши**  
(1789–1857)

Отец численной оптимизации

Открыта дверь к сложным функциям

# Проблема: вычислительная сложность

## Теория vs Практика

- ✓ Работает
- ✗ Полный градиент!

## Сложность

$n$  переменных:

- ▶ Градиент:  $O(n)$

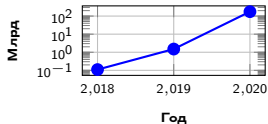
$n = 10^9$ :

- ▶ 1 итер = млрд оп

1847: нет ПК

## Современность

- ▶ GPT-3: 175 млрд
- ▶ LLaMA: 70 млрд



## Решение:

- ▶ Стохастический GD
- ▶ Mini-batch

Идея опередила время на 100 лет

# Задача: найти потерянную планету

1 января 1801 года

## Открытие Цереры

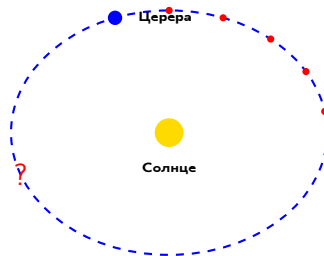
Джузеппе Пиацци открыл  
объект — Церера

## Проблема:

- ▶ Наблюдал 41 ночь
- ▶ Скрылась за Солнцем
- ▶ Где появится?

## Задача:

По 41 наблюдению  
восстановить орбиту



Как найти наилучшие  
параметры орбиты?

Наблюдения зашумлены

# Решение Гаусса

Метод наименьших квадратов, 1801

## Карл Фридрих Гаусс

24 года

### Идея:

Минимизировать  
сумму квадратов  
отклонений

### Формула

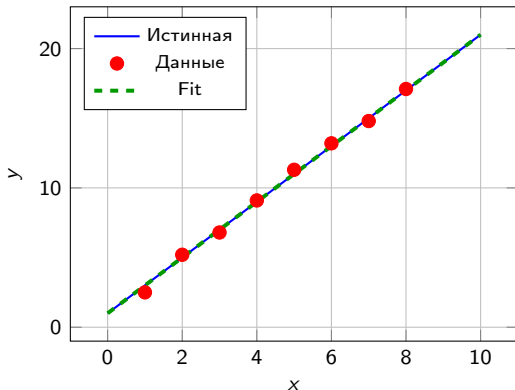
$y_i$  — наблюдения

$f_i$  — модель

$$\theta^* = \arg \min_{\theta} \sum (y_i - f_i)^2$$

### Результат:

Предсказал положение  
Цереры с поразительной



Метод находит линию, минимизирующую  
сумму квадратов вертикальных отклонений



# Почему квадраты?

Вопрос: Почему не  $\sum |y_i - f_i|$ ?

## Причина 1

### Практическая

Квадрат  
дифференцируем  
Модуль — нет

## Причина 2

### Статистическая

Если ошибки  
нормальные  
 $MSE = MLE$

## Причина 3

### Геометрическая

Квадрат =  
евклидово  
расстояние

**Квадраты — обоснованный выбор**

Метод наименьших квадратов = первая задача ML

- ▶ **Данные:**  $(x_i, y_i)$
- ▶ **Модель:**  $f(\theta, x)$
- ▶ **Потери:** MSE
- ▶ **Оптимизация:** найти минимум

## Линейная регрессия

Формула:

$$\theta^* = (X^T X)^{-1} X^T y$$

Аналитическое решение

## Современный ML

Та же структура:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

$L$  — функция потерь

Гаусс заложил фундамент ML в 1801

**Рональд Фишер** — британский статистик

**Ключевая идея:** Оценивать параметры через максимизацию правдоподобия

## Правдоподобие

Вероятность данных  $D$   
при параметрах  $\theta$ :

$$L(\theta) = P(D \mid \theta)$$

## Принцип MLE

Maximum Likelihood  
Estimation:

$$\theta^* = \arg \max_{\theta} L(\theta)$$

**Интуиция:** Выбираем параметры, делающие данные наиболее вероятными

**Связал статистику и оптимизацию**

**Наблюдение:** Максимизация правдоподобия = минимизация потерь

**Шаг 1:** Переход к логарифму

$$\theta^* = \arg \max_{\theta} \log L(\theta)$$

**Шаг 2:** Максимизация = минимизация отрицательного

$$\theta^* = \arg \min_{\theta} ( - \log L(\theta) )$$

**Результат:** Это функция потерь!

**Статистическая оценка = оптимизация**

# Пример: логистическая регрессия

## Постановка:

$y_i \in \{0, 1\}$  — метки

$$p_i = \sigma(\theta^T x_i)$$

## Правдоподобие:

$$L = \prod p_i^{y_i} (1 - p_i)^{1-y_i}$$

## Отрицательный логарифм:

$$\begin{aligned} -\log L = & -\sum [y_i \log p_i \\ & + (1 - y_i) \log(1 - p_i)] \end{aligned}$$

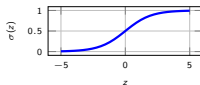
## Результат

Это binary cross-entropy!

$$\text{BCE} = -\sum \ell_i$$

где

$$\ell_i = y_i \log p_i + (1 - y_i) \log(1 - p_i)$$



MLE даёт правильную функцию потерь

Фишер показал

Статистическая оценка = Оптимизация

Оценить параметры



Максимизировать правдоподобие



Минимизировать потери

**Мост между статистикой и оптимизацией**

## Что мы узнали

1. **Коши (1847):** Градиентный спуск — итеративная оптимизация
2. **Гаусс (1801):** Метод наименьших квадратов — первая задача ML
3. **Фишер (1920-е):** MLE — связь статистики и оптимизации

## Главная идея

Обучение модели = Задача оптимизации

**Далее:** Backpropagation и революция нейросетей

# Проблема многослойных сетей

1960-е годы

## Персептрон (1958)

Работает, но ограничен

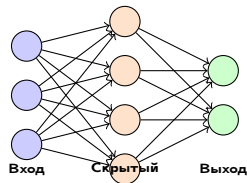
**XOR не решается!**

## Мински-Паперт (1969):

“Персептрон ограничен”

Финансирование закрыто

## Многослойные сети



## Вопрос:

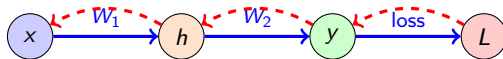
Как обучать скрытый слой?

**Нужен способ вычислять градиенты**



# Что такое backpropagation?

Идея: Цепное правило в обратном направлении



Forward pass (синий)

Backward pass (красный)

**Forward:**  $h = \sigma(W_1 x)$ ,  $y = \sigma(W_2 h)$ ,  $L = \text{loss}(y, y_{\text{true}})$

**Backward:**  $\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial W_2}$ ,  $\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial h} \cdot \frac{\partial h}{\partial W_1}$

Ошибка распространяется назад

**Сеппо Линнайрмаа**  
Финский математик

**Магистерская работа (1970):**  
Изобрёл автоматическое  
дифференцирование

**Reverse mode =**  
= backpropagation

**Проблема:**  
Работа на финском языке

**Автодифф vs Символьное**

**Символьное:**

$$f(x) = x^2 + 3x$$

$$f'(x) = 2x + 3$$

Аналитическая формула

**Автодифф:**

Вычисляем  $f(x)$

Сохраняем промежуточные

Идём назад

Получаем  $f'(x)$

**Преимущество:**

Работает для любой программы!

Изобрёл, но мир не узнал

# Пол Вербос: забытая диссертация

1974, Гарвард

## Пол Вербос

PhD студент, Гарвард

## Диссертация (1974):

“Beyond Regression...”

## Что сделал:

Описал backpropagation

Применил к нейросетям

Показал, как обучать

многослойные сети

## Проблема:

Не опубликовал в журнале

## Цитата Вербоса:

*“Я думал, что это очевидно. Я не понимал, почему никто не использует это.”*

## Почему не заметили:

Диссертация — не статья

“Зима нейросетей”

Финансирование закрыто

Все переключились на

символьный AI

## Ирония:

В 1986 переоткрыли заново

Никто не читал 12 лет

# Триумф: Rumelhart, Hinton, Williams

1986, Nature

## Статья в Nature (1986)

“Learning representations  
by back-propagating errors”

### Авторы:

David Rumelhart  
Geoffrey Hinton  
Ronald Williams

### Что сделали:

Переоткрыли backpropagation  
Подали как “универсальный”  
алгоритм  
Показали примеры  
Опубликовали в Nature

### Почему взлетело:

**Время:** Компьютеры  
стали мощнее

**Подача:** Ясное объяснение

**Примеры:** Работающий код

**Авторитет:** Nature

### Результат:

Backprop стал стандартом  
Вторая волна интереса к NN  
Hinton — “крёстный отец”  
глубокого обучения

Мир наконец узнал

# Почему не взлетел сразу?

1990–2010: тёмные годы

## Проблема 1:

Затухающие градиенты

Hochreiter (1991)

Сигмоида:  $\sigma'(x) \leq 0.25$   
10 слоёв:  $0.25^{10} \approx 10^{-6}$

## Проблема 2:

Переобучение

Мало данных

Нет регуляризации

## Проблема 3:

Вычисления

CPU медленные

Нет GPU для ML

Обучение занимает недели

## Триумф SVM (1995–2010):

Support Vector Machines

Выпуклая оптимизация

Kernel trick

Работают “из коробки”

SVM побеждают нейросети

Не хватало данных и GPU

# Что изменилось к 2012?

Ренессанс глубокого обучения

## 1. Данные

ImageNet (2009)

1.2 млн изображений

1000 классов

Публичный датасет

## 2. Вычисления

GPU для ML

CUDA (2007)

Ускорение в 10–100 раз

Обучение за дни

## 3. Архитектурные трюки

**ReLU (2010):**

$$f(x) = \max(0, x)$$

Нет затухания градиентов

**Dropout (2012):**

Случайное выключение

Регуляризация

## 4. AlexNet (2012)

Hinton + студенты

Победа на ImageNet

16% → 10% ошибки

**Начало новой эры**

**Данные + GPU + ReLU + Dropout**

## История:

1970: Линнайнмаа  
(не заметили)

1974: Вербос  
(не прочитали)

1986: Rumelhart-Hinton-Williams  
(триумф!)

1990–2010: Проблемы

2012: AlexNet и новая эра

## Главная идея:

Backprop = цепное правило  
задом наперёд

## Почему важно:

Без backprop нет  
современного DL

Фундамент для:  
CNN, RNN, Transformer,  
Diffusion models

**Далее: SGD, Momentum, Adam**

# Чистый SGD

Стохастический градиентный спуск

## Проблема GD:

Полный градиент = дорого!

$$\nabla L = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i$$

Нужно пройти по **всем** данным

## Идея SGD:

Использовать градиент  
по **одному** примеру

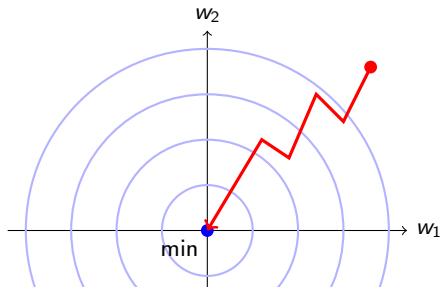
$$\nabla L \approx \nabla \ell_i$$

Шумно, но быстро!

## Mini-batch SGD:

Компромисс: градиент  
по **небольшой пачке** (batch)

$$\nabla L \approx \frac{1}{B} \sum_{i=1}^B \nabla \ell_i$$





# Проблемы “голого” SGD

## 1. Плохая обусловленность

Ландшафт потерь вытянут в одних направлениях и сжат в других

## 2. Зигзаги в узких долинах

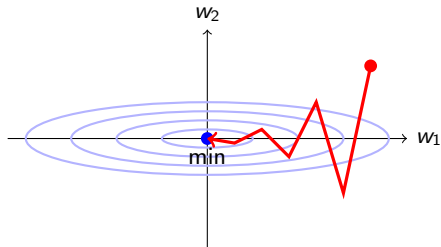
Градиент “скачет” по стенкам долины

Медленное движение вдоль дна

## 3. Чувствительность к $\eta$

Один  $\eta$  для всех параметров

Для одних — слишком большой, для других — слишком маленький



Красный: траектория SGD  
в узкой долине

**SGD неэффективен в сложных ландшафтах**

# Momentum

Борис Поляк, 1964

## Интуиция:

Шарик с массой катится  
по ландшафту потерь

Накапливает инерцию

## Что это даёт:

### Сглаживание шума:

Инерция гасит  
случайные колебания

### Ускорение по долинам:

Инерция помогает  
двигаться вдоль дна

## Формула:

$$g_t = \nabla L(\theta_t)$$

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

$\beta$  (momentum):

Коэффициент инерции  
(обычно 0.9)

Синий: траектория Momentum

**Momentum = SGD + инерция**

# Nesterov Accelerated Gradient (NAG)

Юрий Нестеров, 1983

## Контекст:

Выпуклая оптимизация

Теоретически оптимальный  
метод первого порядка

## Идея:

“Заглядывание вперёд”

Сначала делаем шаг  
по инерции, потом  
считаем градиент

## Momentum:

$$g_t = \nabla L(\theta_t)$$

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

## Nesterov:

$$\theta_{\text{lookahead}} = \theta_t - \alpha \beta v_{t-1}$$

$$g_t = \nabla L(\theta_{\text{lookahead}})$$

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

## Практика:

Иногда работает лучше  
обычного Momentum

**NAG = Momentum + “заглядывание вперёд”**

### **SGD:**

Просто, но неэффективно

Зигзаги в узких долинах

### **Momentum:**

Добавляет инерцию

Сглаживает шум

Ускоряет сходимость

### **Nesterov:**

“Заглядывает вперёд”

Иногда ещё лучше

### **Связь с будущим:**

Momentum — это прототип

“первого момента” в Adam

### **Ограничения:**

Всё ещё один learning rate  
для всех параметров

Не решает проблему  
разных масштабов

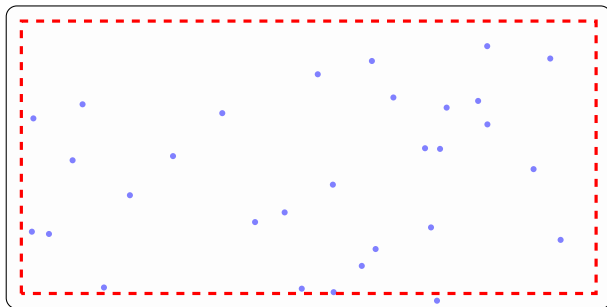
### **Что дальше?**

Адаптивные методы!

Далее: Adagrad, RMSProp, Adam

Проблема: Полный градиент — это медленно

Полный датасет ( $N = 1,000,000$ )



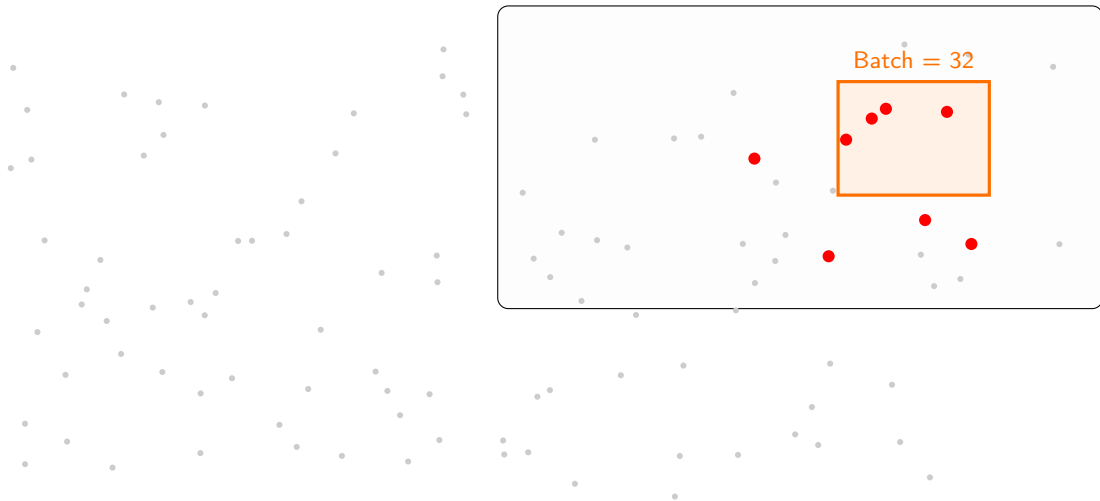
Используем ВСЕ данные

## Решение: Стохастический градиент (SGD)

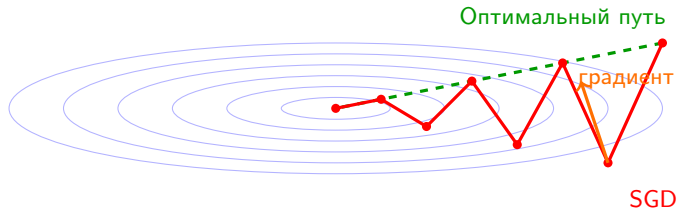
Полный датасет ( $N = 1,000,000$ )



Полный датасет ( $N = 1,000,000$ )



## Новая проблема SGD: зигзаги в узких долинах

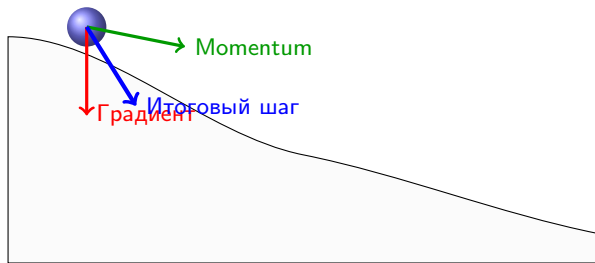


### Проблема

Градиент указывает поперёк долины, а не вдоль неё — много лишних шагов!

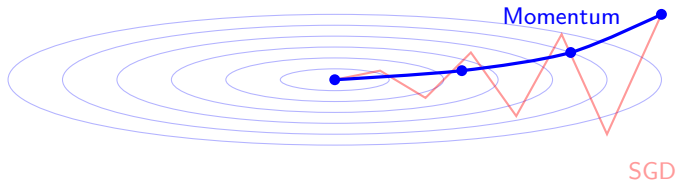


## Решение: Momentum (Инерция)



### Идея

Добавляем "инерцию" как у шара на склоне — сглаживает зигзаги и ускоряет движение.



## Формула

$$v_t = \beta v_{t-1} - \alpha \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t + v_t$$

где  $\beta \approx 0.9$

## Эффект

Накопленная скорость  $v_t$  гасит колебания и ускоряет движение вдоль долины.

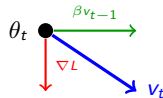
# Улучшение: Nesterov Momentum (NAG)

## Обычный Momentum

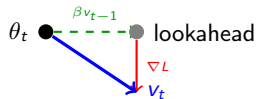
1. Градиент в  $\theta_t$
2. Обновление скорости
3. Шаг

$$v_t = \beta v_{t-1} - \alpha \nabla L(\theta_t)$$

## Обычный Momentum



## Nesterov Momentum



## Nesterov Momentum

1. "Заглядываем вперёд"
2. Градиент в будущей точке
3. Шаг

$$v_t = \beta v_{t-1} - \alpha \nabla L(\theta_t + \beta v_{t-1})$$

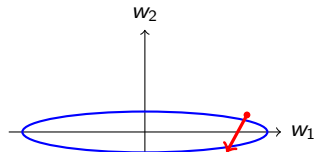
### Что решили

- ▶ **Скорость:** Mini-batch вместо полного GD
- ▶ **Зигзаги:** Momentum сглаживает траекторию

### Что осталось

- ▶ Единый learning rate  $\alpha$  для всех параметров
- ▶ Проблема разных масштабов

### Проблема масштаба



Один  $\alpha$  не подходит  
для обоих параметров

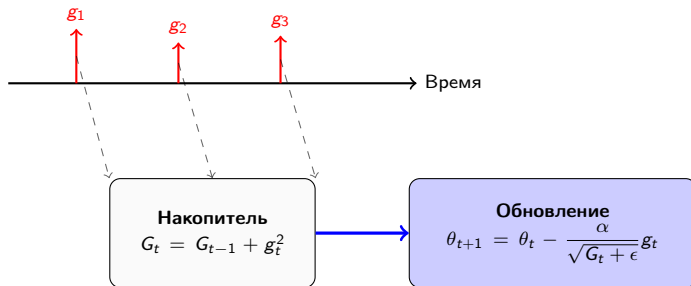
### Следующий шаг

Нужны методы с **адаптивным** learning rate для каждого параметра: AdaGrad, RMSprop, Adam.

### Интерпретация

Один и тот же размер шага  $\alpha$  не может быть одновременно хорош для быстрого (горизонталь) и медленного (вертикаль) направлений.

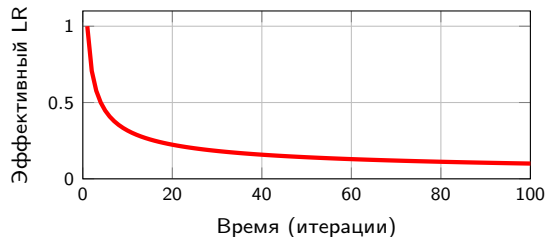
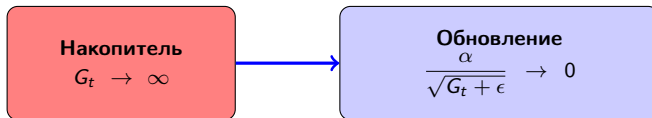
## Решение 1: Adagrad (Adaptive Gradient)



### Интерпретация

Накапливаем квадраты всех прошлых градиентов ( $G_t = \sum_{i=1}^t g_i^2$  покомпонентно). Чем больше накоплено, тем **меньше** становится шаг для этого параметра.

## Проблема Adagrad: "умирающий" learning rate

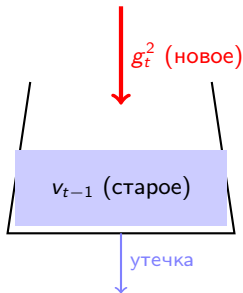


### Интерпретация

Накопленная сумма  $G_t$  только растёт. Рано или поздно шаг становится настолько маленьким, что обучение останавливается.

## Решение 2: RMSProp

Geoffrey Hinton, лекция 6.5, 2012



Экспоненциальное скользящее среднее

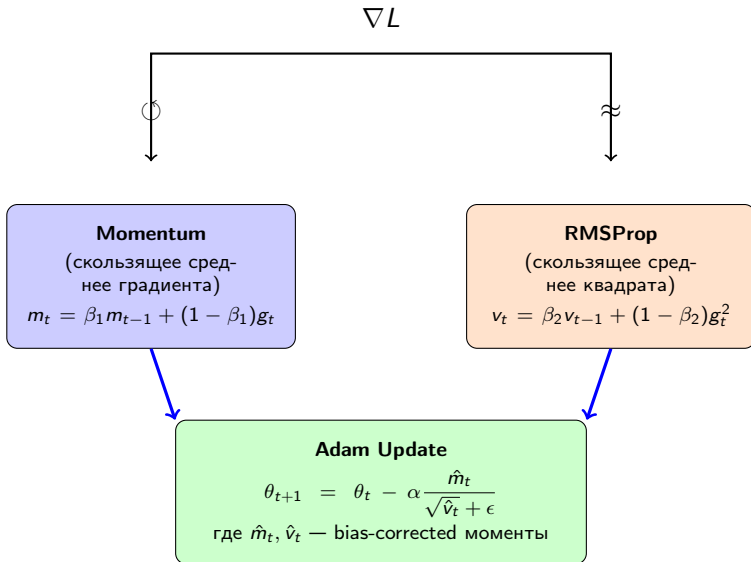
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

### Интерпретация

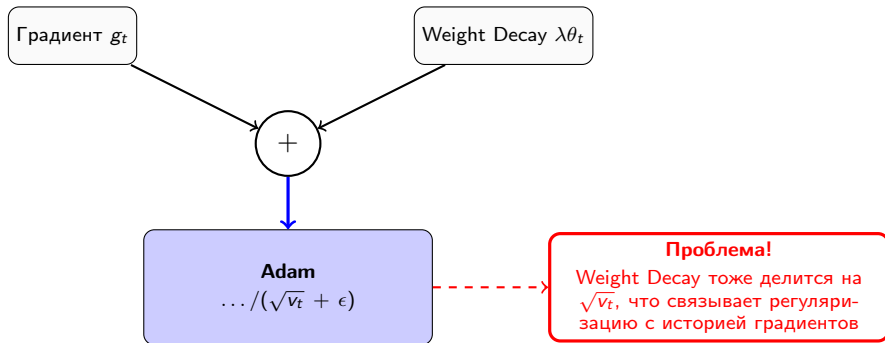
Вместо бесконечного накопления используем "дырявое ведро": постоянно добавляем новое и "забываем" часть старого (обычно  $\beta_2 = 0.9$  или  $0.99$ ).



## Синтез: Adam (Adaptive Moment Estimation)



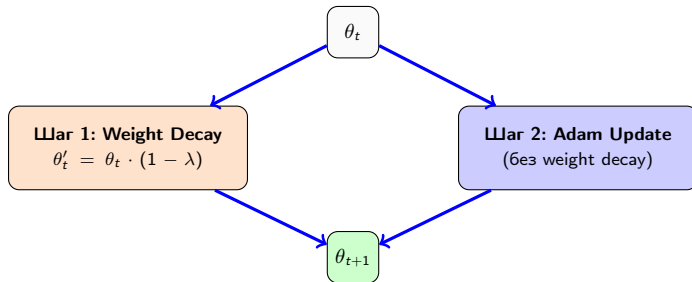
## Скрытая проблема Adam: неправильный Weight Decay



### Интерпретация

В стандартном Adam регуляризация применяется **до** адаптивного масштабирования. Параметры с большой историей градиентов получают меньше регуляризации, что нарушает её смысл.

### Решение 3: AdamW (Decoupled Weight Decay)



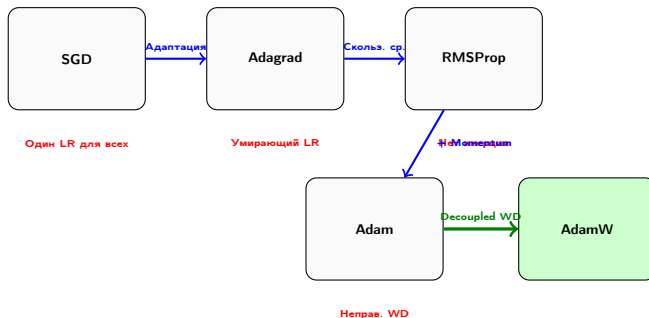
Полная формула

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \alpha \lambda \theta_t$$

#### Интерпретация

Weight Decay применяется **независимо** от адаптивного масштабирования Adam, что восстанавливает правильную L2-регуляризацию.

## Итоги: путь к AdamW



### Вывод

AdamW = Momentum + RMSProp + правильный Weight Decay.  
Стандарт для большинства задач.

# Почему поиск продолжается после AdamW?

## Проблема 1: Память

AdamW хранит два дополнительных массива для каждого параметра:

- ▶  $m_t$  (первый момент, momentum)
- ▶  $v_t$  (второй момент, RMSProp)

**Итог:** Память на параметры  $\times 3$ .  
Для GPT-3 (175B параметров):  
 $175 \times 10^9 \times 4 \text{ bytes} \times 3 = 2.1 \text{ TB!}$

## Проблема 2: Эффективность

AdamW — метод первого порядка:

- ▶ Использует только градиент  $\nabla L$
- ▶ "Слеп" к кривизне (Гессиану)
- ▶ Может быть неэффективен на сложных поверхностях

## Ограничения AdamW

**Память  $\times 3$**

$\theta, m_t, v_t$

**Только 1-й порядок**

Не видит кривизну



Поиск более эффективных методов

# Lion (EvoLved Sign Momentum)

Google Brain, 2023

## Ключевая идея

Использовать только **знак** накопленного momentum, отказавшись от адаптивного масштабирования (второго момента).

## Алгоритм

1. Вычисляем градиент  $g_t$
2. Интерполируем:  
$$c_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
3. Шаг по знаку:  
$$\theta_{t+1} = \theta_t - \alpha \cdot \text{sign}(c_t)$$
4. Обновляем momentum:  
$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t$$
  
где  $\beta_1 = 0.9$ ,  $\beta_2 = 0.99$

## Сравнение шага

Adam:


$$\alpha \frac{m_t}{\sqrt{v_t + \epsilon}}$$

Величина зависит от  $v_t$

Lion:


$$\alpha \cdot \text{sign}(c_t)$$

Величина всегда  $\alpha$

## Преимущества и недостатки

**Память:** Только  $m_t \rightarrow$  память  $\times 2$

**Эффективность:** Часто быстрее AdamW на ViT

**Стабильность:** Требуется меньший lr

**Новизна:** Меньше опыта применения

# Sophia (Second-order Clipped Optimization)

Stanford, 2023

## Ключевая идея

Приблизить метод Ньютона (второго порядка), используя **дешёвую оценку диагонали Гессiana**.

## Алгоритм

1. Вычисляем градиент  $g_t$
2. Обновляем momentum:  
 $m_t = \beta m_{t-1} + (1 - \beta)g_t$
3. Оцениваем диагональ Гессiana  $h_t$  (раз в  $k$  шагов)

4. **Шаг с клиппингом:**  
 $\theta_{t+1} = \theta_t - \alpha \cdot \text{clip}\left(\frac{m_t}{\gamma h_t + \epsilon}, -1, 1\right)$   
где  $\gamma = 0.01$  — гиперпараметр

## Адаптация к кривизне



## Преимущества и недостатки

**Скорость:** В  $2\times$  быстрее AdamW на LLM

**Теория:** Использует информацию о кривизне

**Память:** Хранит  $h_t \rightarrow$  память  $\times 3$

## Сравнение: AdamW vs Lion vs Sophia

| Характеристика | AdamW      | Lion        | Sophia            |
|----------------|------------|-------------|-------------------|
| Память         | $\times 3$ | $\times 2$  | $\times 3$        |
| Что хранит     | $m_t, v_t$ | $m_t$       | $m_t, h_t$        |
| Порядок метода | 1-й        | 1-й         | 2-й (приближённо) |
| Адаптация шага | По $v_t$   | Нет (знак)  | По кривизне $h_t$ |
| Стабильность   | Высокая    | Средняя     | Средняя           |
| Статус         | Стандарт   | Эксперимент | Эксперимент       |

### Когда что использовать?

- ▶ **AdamW:** Выбор по умолчанию. Надёжно и проверено временем.
- ▶ **Lion:** Если ограничены в памяти (особенно на Vision Transformers).
- ▶ **Sophia:** Если обучаете LLM с нуля и хотите ускорить сходимость.

### Главный вывод

Поле активно развивается. Появляются новые идеи, которые бросают вызов AdamW.

Однако на 2024 год **AdamW остаётся самым надёжным и универсальным выбором.**

**Далее:** Методы, учитывающие структуру параметров (Shampoo, Muon)



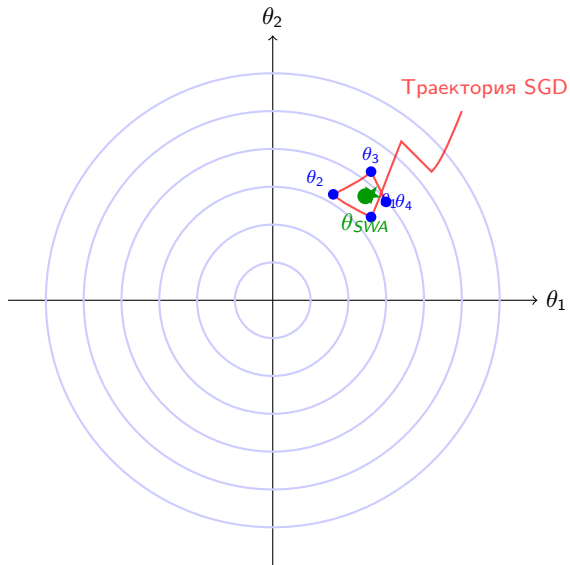
## Проблема: не все минимумы одинаково полезны



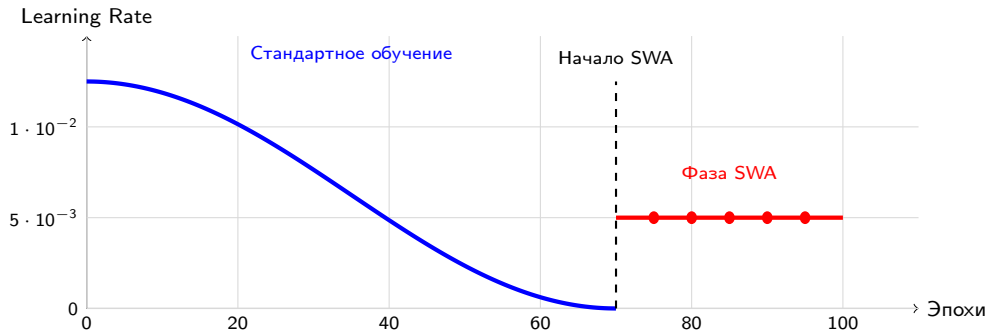
### Интерпретация

Небольшое смещение данных (Train  $\rightarrow$  Test) может "выбросить" нас из узкого минимума, но не из широкого. **Широкие минимумы лучше обобщаются.**

## Идея SWA: усреднение весов в конце обучения



# Как работает SWA на практике



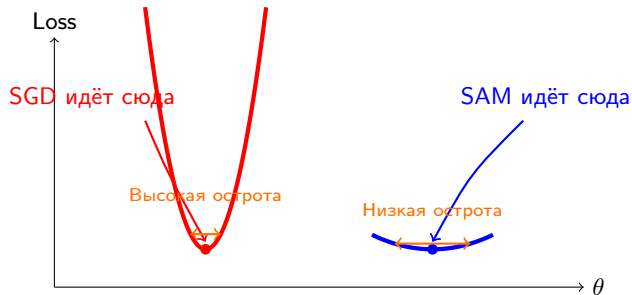
## Процедура

1. Обучаем модель с обычным спадающим LR.
2. Включаем SWA: высокий постоянный LR.
3. Собираем чекпоинты модели. Усредняем собранные веса.

## Преимущества

**Дешево**  
**Эффективно**  
**Универсально**

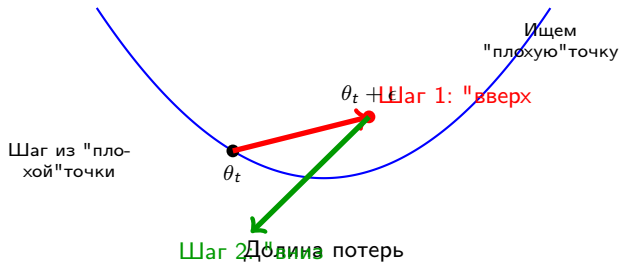
# Идея SAM: минимизация "остроты"



## Интерпретация

SAM ищет самую **плоскую область**, а не самую низкую точку. Он активно избегает "острых" минимумов.

# Как работает SAM: шаг "вверх шаг "вниз"



## Процедура

1. **Шаг "вверх"**: Найти "плохую" точку в окрестности.
2. **Шаг "вниз"**: Градиент в "плохой" точке.

## Компромисс

**Дорого:**  $\times 2$  медленнее

**Эффективно**

|             | SWA                     | SAM                    |
|-------------|-------------------------|------------------------|
| Идея        | Усреднить веса в конце  | Искать плоские регионы |
| Когда?      | После обучения          | Во время обучения      |
| Стоимость   | Почти бесплатно         | ×2 к времени           |
| Что меняет? | Только финальную модель | Весь процесс           |
| Аналогия    | Найти центр плато       | Избегать острых пиков  |

### Практика

- ▶ **SWA:** Легко попробовать, почти всегда даёт плюс.
- ▶ **SAM:** Если готовы платить вдвое дороже.

### Главный вывод

Управление геометрией минимума — мощный инструмент.

Можно комбинировать: AdamW + SAM, AdamW + SWA.

Далее: Практическое руководство

# Проблема AdamW: "слепота" к структуре

## Как работает AdamW:

Рассматривает матрицу весов  $W$  как "мешок" независимых чисел.

### Что это значит:

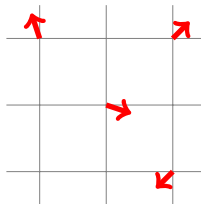
- ▶ Каждый вес  $w_{ij}$  получает свой собственный адаптивный 'learning rate'.
- ▶ Игнорируются корреляции между строками и столбцами.

## Интерпретация

Представьте, что вы настраиваете эквалайзер. AdamW двигает каждый ползунок независимо. Но частоты сильно связаны, и двигать их нужно согласованно.

## Независимые обновления

Матрица весов  $W$



Каждый вес обновляется в своём собственном направлении, игнорируя соседей.

# Идея Shampoo: адаптация по строкам и столбцам

## Ключевая идея:

Рассматривать параметры как матрицы/тензоры, а не как набор чисел.

## Как это работает:

- ▶ Вычисляем статистику для **целых строк**.
- ▶ Вычисляем статистику для **целых столбцов**.
- ▶ Используем их для "сжатия" градиента.

## Интерпретация

Вместо настройки каждого музыканта в оркестре (AdamW), мы настраиваем громкость целых секций: сначала всех скрипок, потом всех труб. Это более согласованно.

## Структурные обновления



Обновления происходят согласованно по целым осям тензора.



# Как работает Shampoo: предобуславливание

Идея: "Выпрямить" пространство

Shampoo вычисляет матрицы  $L$  и  $R$ , которые "выпрямляют" ландшафт потерь.

Формула (концептуально):

$$\text{Обновление} = L^{-1/2} \cdot G \cdot R^{-1/2}$$

Где  $G$  — градиент.

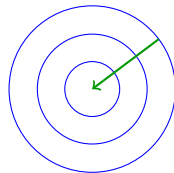
## Интерпретация

Shampoo как бы надевает "волшебные очки" на градиентный спуск. В этих очках вытянутый овраг выглядит как идеально круглая яма, и спускаться по ней становится тривиально — просто по прямой к центру.

"Выпрямление" ландшафта



Обычный спуск (зигзаги)



Спуск в "выпрямленном" пространстве

# Идея Muon: адаптация по "силе" слоя

## Ключевая идея:

Контролировать не отдельные веса, а "силу" всего слоя.

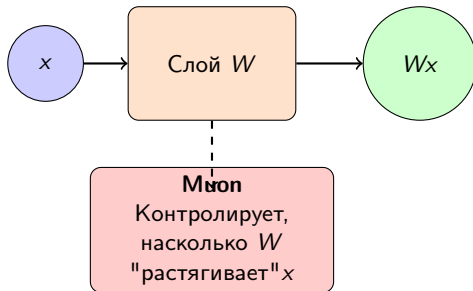
## Что такое "сила" слоя?

Это его способность "растягивать" входной вектор (математически — спектральная норма или максимальное сингулярное число).

## Интерпретация

Представьте, что каждый слой — это усилитель. Muon следит не за каждой деталью усилителя, а за его общей "ручкой громкости". Он адаптирует 'learning rate' так, чтобы эта "громкость" менялась плавно.

## Контроль "растяжения"



## Сравнение: AdamW vs Shampoo vs Muon

|                 | AdamW                       | Shampoo                    | Muon                       |
|-----------------|-----------------------------|----------------------------|----------------------------|
| Что адаптирует? | Каждый вес независимо       | Строки и столбцы матрицы   | "Силу" всего слоя (спектр) |
| Аналогия        | Настройка каждого музыканта | Настройка секций оркестра  | Настройка общей громкости  |
| Сложность       | Низкая                      | Высокая                    | Средняя                    |
| Где лучше       | Универсальный               | Очень большие модели (TPU) | Трансформеры (LLM)         |

### Практика

- ▶ **AdamW:** Ваш выбор по умолчанию.
- ▶ **Shampoo:** Если у вас есть доступ к большим кластерам TPU (Google).
- ▶ **Muon:** Очень перспективен для обучения Трансформеров. Часто используется в комбинации с AdamW.

### Главный вывод

Будущее за оптимизаторами, которые "понимают" структуру параметров и адаптируются к ней.

Простая поэлементная адаптация (как в AdamW) — уже не предел.